# NeoLoad: Tricentis Performance testing Tool

### What is NeoLoad?

**NeoLoad** is an enterprise-grade **performance testing tool** by Tricentis, designed to support continuous load testing for **web, mobile, and API** apps. It focuses on scalability, CI/CD integration, real-time diagnostics, and ease of use for performance engineers.

## NeoLoad – Key Pros

| Area | Advantages |
|---|---|
| **Ease of Use** | GUI-driven design; quick test creation without scripting. Great for less technical testers. |
| **Dynamic Infrastructure** | Native integration with **Kubernetes**, **Docker**, and **cloud load generators**. |
| **Test Design Efficiency** | Record-and-playback + parameterization of transactions. Excellent for UI-heavy flows. |
| **Real-Time Monitoring** | In-depth analytics with built-in monitoring for servers, infrastructure, DBs, and apps. |
| **DevOps Integration** | Integrates with **Jenkins**, **GitHub Actions**, **Azure DevOps**, **Dynatrace**, **Prometheus**, etc. |
| **Correlation Engine** | Smart correlation to auto-handle session IDs, tokens, etc. (more powerful than JMeter). |
| **SLA Assertions & Reporting** | Built-in SLA management, pass/fail criteria, and actionable dashboards. |
| **Protocol Coverage** | Wide: HTTP/S, WebSocket, Citrix, SAP, Oracle, MQTT, etc. |
| **Enterprise Collaboration** | Role-based access, version control, shared resources, and integration with **Tricentis Tosca** for end-to-end test strategy. |

## NeoLoad – Key Cons

| Area | Disadvantages |
|---|---|
| **Cost** | **Commercial license required** (expensive compared to JMeter). Tiered by VU and usage model. |
| **Learning Curve for Scripting Pros** | Power users coming from JMeter or Gatling may find it **limiting for custom scripting**. |
| **Limited Community** | Smaller open-source and community ecosystem compared to JMeter. |
| **Less Lightweight** | Heavier setup footprint and resource consumption for local runs. |
| **Limited Taurus-Like Flexibility** | Cannot be as easily coded/templated via YAML like Taurus CLI/CI pipelines. |

## NeoLoad vs JMeter + Taurus + BlazeMeter – Detailed Comparison

| Feature / Area | NeoLoad | JMeter | Taurus + BlazeMeter |
|---|---|---|---|
| **License** | Commercial (expensive) | Open-source | Open-source + SaaS |
| **Ease of Use (UI)** | Intuitive GUI | Steep learning curve | CLI-first, not beginner-friendly |
| **Custom Logic & Scripting** | Limited (mostly GUI-driven) | Full scripting flexibility | YAML scripting, Python hooks |
| **Protocol Coverage** | Wide | Wide | Based on JMeter/Gatling |
| **Test as Code Support** | (limited YAML) | With effort | YAML-first |
| **Smart Correlation** | Built-in AI engine | Manual regex mostly | JMeter-dependent |
| **Monitoring & Diagnostics** | Native + APM integrations | Plugins required | BlazeMeter provides integrated APM |
| **CI/CD Integration** | Native Azure, Jenkins, GitHub | Via plugins/scripts | Strong via CLI/REST API |
| **Cloud Load Generation** | Built-in cloud infra | Not native | BlazeMeter cloud engines |
| **Reporting & SLA Management** | Advanced, visual | CSV/XML, plugins needed | Real-time in BlazeMeter UI |
| **Team Collaboration / Scalability** | Versioning, roles, API | Single-user focused | SaaS & Git-integrated |

## When to Choose NeoLoad

Recommended **if**:

- You want to **scale performance testing across QA orgs** (not just DevOps/engineers).
- You value **rapid test creation**, smart correlation, and **SLA-based pass/fail dashboards**.
- You need **strong integrations with enterprise tools** (Dynatrace, ServiceNow, Azure).
- You're testing **complex enterprise protocols** (Citrix, SAP, MQTT, etc.).
- You have budget for enterprise tooling and need **centralized visibility**.

## When NOT to Choose NeoLoad

Avoid if:

- Your team is **already proficient in JMeter/Taurus** and seeks **test-as-code + full control**.
- You require **high customization, scripting**, and edge case control (e.g., custom headers, encryption, dynamic sessioning).
- You are budget-conscious and have effective open-source CI/CD already in place.
- You run **heavily containerized or cloud-native microservices** and prefer CLI-first YAML (Taurus) or API-first scripting.

## Recommendations for Ascensus

- **Keep using JMeter + Taurus + BlazeMeter** as your base toolchain — especially since you already have:
  - Taurus YAML configurations
  - Blazemeter dashboards
  - JMeter scripts for legacy systems
  - Budget-sensitive performance infra
- **Evaluate NeoLoad selectively** for:
  - High-visibility business-critical flows where visual reporting and SLA enforcement are vital.
  - Teams lacking deep JMeter skills but needing to own performance.
  - Replacing expensive 3rd-party monitoring tools (e.g., if NeoLoad can eliminate them).

## Final Verdict

| Factor | Recommendation |
|---|---|
| **Primary Tooling** | Stick with **JMeter + Taurus + BlazeMeter** for flexibility, scalability, and cost-efficiency. |
| **NeoLoad Usage** | Consider for **pilot evaluation** or **POC** on new apps or high-complexity use cases if budget allows. |
| **Best Practice** | Use NeoLoad for **SLA dashboards**, monitoring integration, and **non-technical QA teams** if they must run perf tests independently. |